Task 2
Student Information Dashboard Using React, Angular and Vue

React:

1. Need For React:

Before React:

- full page reloads
- difficult DOM manipulation
- messy frontend code

React introduced:

- reusable components
- faster updates
- declarative UI

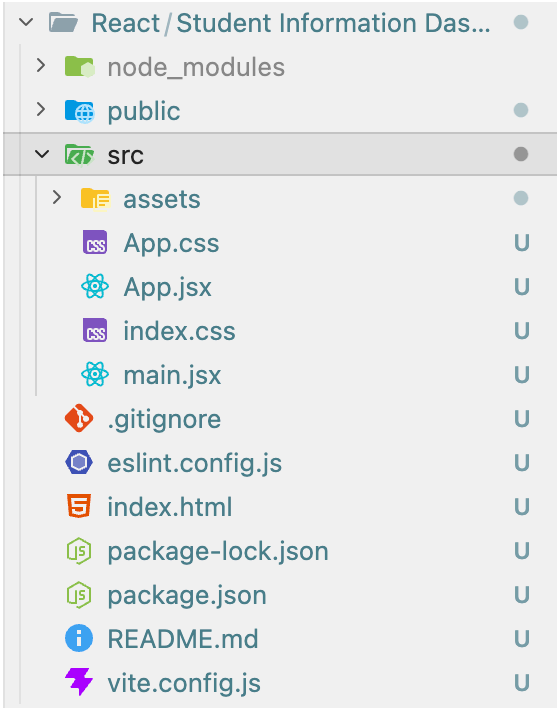
2. Creating react project:

npm create vite@latest

*Vite is a modern frontend build tool faster than Create React App.

```
krishnaik@Krishs-MacBook-Air React % npm create vite@latest

> npx
> create-vite

|
◇  Project name:
|  Student Information Dashboard
|
◇  Package name:
|  student-information-dashboard
|
◇  Select a framework:
|  React
|
◇  Select a variant:
|  JavaScript
|
◇  Install with npm and start now?
|  Yes
|
◇  Scaffolding project in /Users/krishnaik/Downloads/IncubXperts Internship/Task 2/React/Student Information Dashboard...
|
◇  Installing dependencies with npm...

added 135 packages, and audited 136 packages in 11s

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
|
◇  Starting dev server...

> student-information-dashboard@0.0.0 dev
> vite


  VITE v8.0.13  ready in 622 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

3. Folder Structure



• **node_modules:** node_modules stores all dependencies required by the project.

• **public**: stores static assets that you want the web server to serve directly without being processed by your build tool

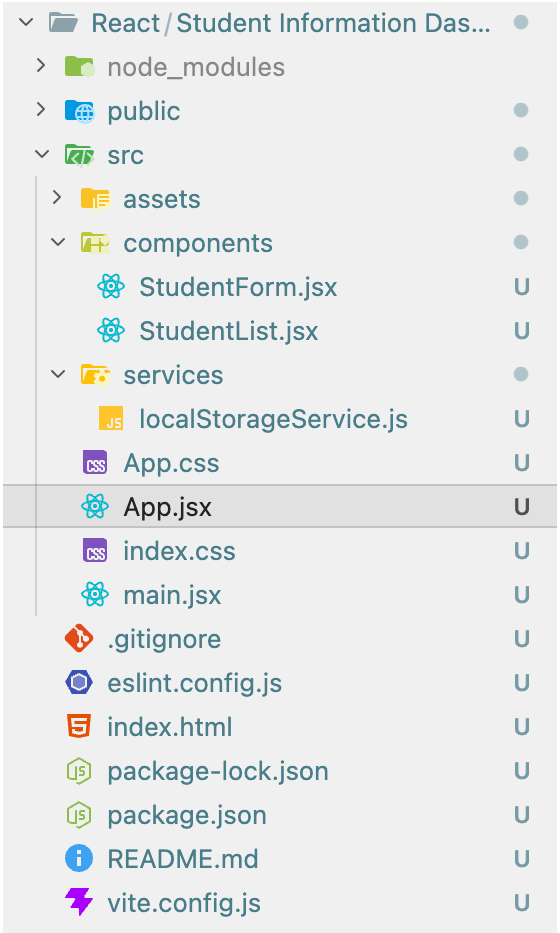
• **src**: This is the main working directory where application logic and components are created.

• **package.json**: stores project metadata file.

• **package-lock.json**: locks exact package versions to ensure ensures same installation everywhere

• **vite.config.js**: used for vite configuration, build customization and plugins.

4. Designing Our Project

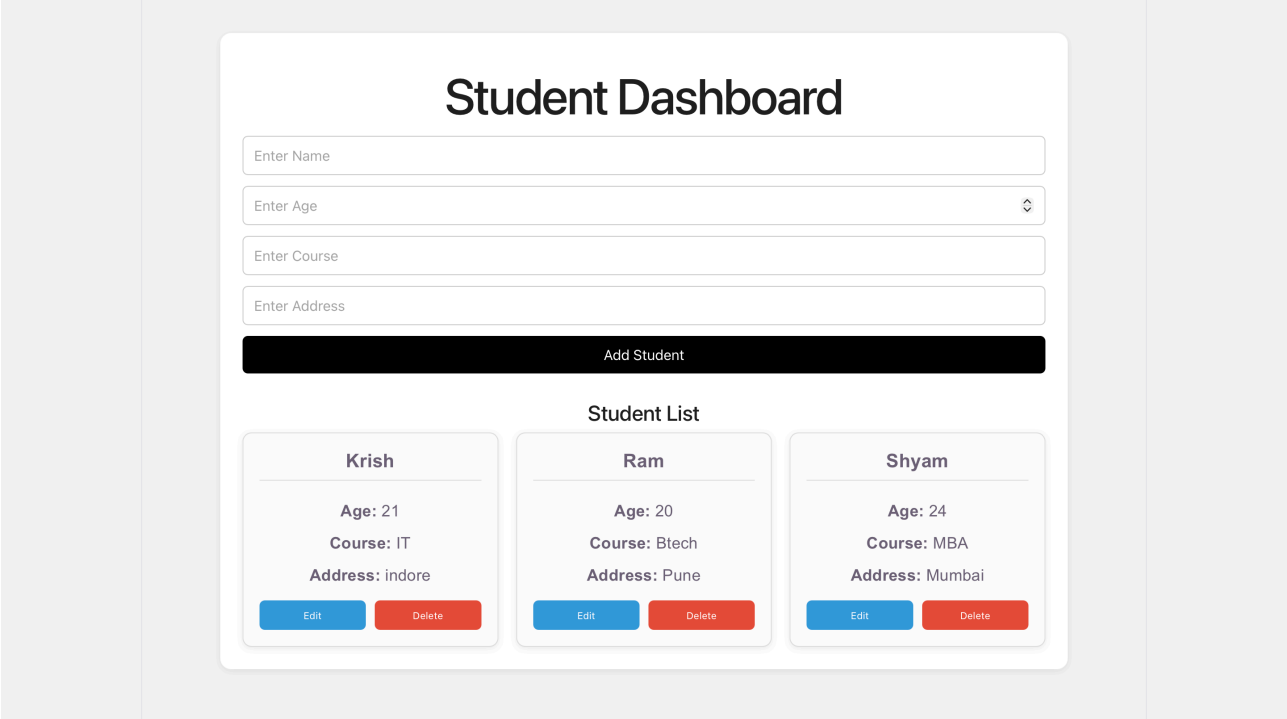


Cleaned the code in App.jsx and App.css and implement our project structure:

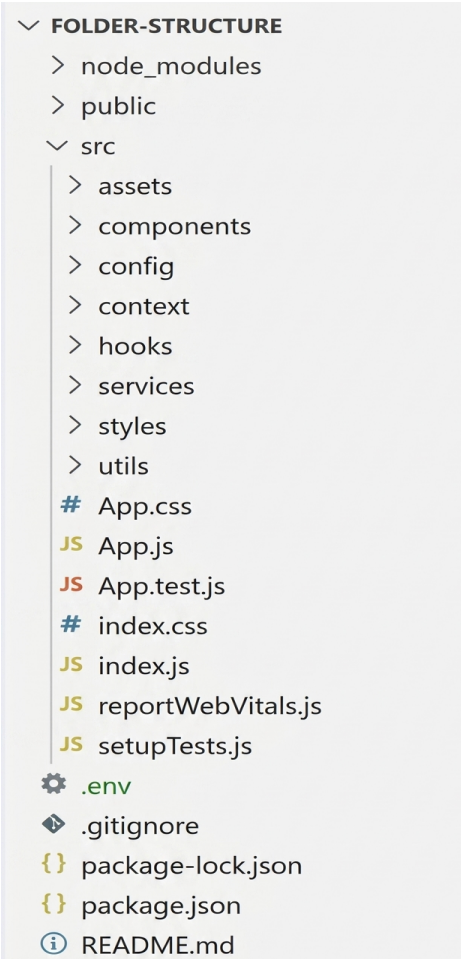
• **components**:
-StudentForm.jsx
-StudentList.jsx

• **services**:
-localStorageService.js

5. Project Screenshot



6. React Folder Architecture in detail



• **components**: A Component is a core building block of React, similar to a JavaScript function, that returns HTML and helps break the UI into manageable pieces.

• **context**: The Context directory manages global state in React, allowing components to share data without prop drilling.

• **hooks**: Hooks enable functional components to use state and React features without classes, promoting reusable logic.

• **services**: The Services directory manages API calls and external interactions, keeping code organized and maintainable.

• **utils**: The Utils directory stores reusable utility functions used across components to centralize common logic.

• **assets**: The Assets folder stores static files like images, icons, and fonts, keeping the project organized.

• **config**: The Config folder holds configuration files like config.js for managing environment settings and build tools.

Angular

1. Need for Angular:

Before Angular:

- Manual DOM manipulation
- No proper structure
- No component reusability
- State management problems

Angular Introduced:

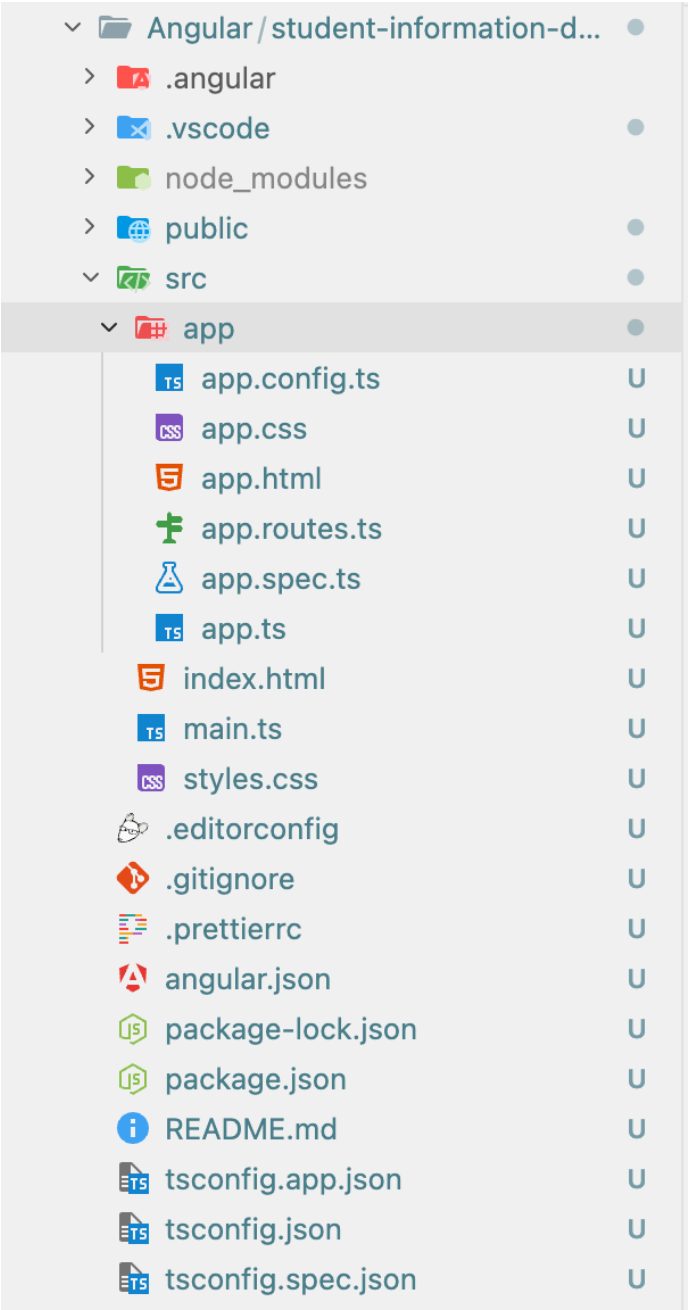
- CLI for structured development
- Two way binding for better synchronization
- Directives for dynamic templates and components for modular UI
- Dependency injection for reusable services

2. Creating Angular Project

npm install -g @angular/cli
ng new student-information-dashboard

```
krishnaik@Krishs-MacBook-Air Angular % npm install -g @angular/cli

added 288 packages in 29s

70 packages are looking for funding
  run `npm fund` for details
krishnaik@Krishs-MacBook-Air Angular % ng new student-information-dashboard
 Would you like to enable autocompletion? This will set up your terminal so pressing TAB while typing Angular CLI commands will show possible options and autocomplete arguments. (Enabling autocompletion will modify configuration files in your home directory.) Yes
Appended `source <(ng completion script)` to `/Users/krishnaik/.zshrc`. Restart your terminal or run the following to autocomplete `ng` commands:

    source <(ng completion script)

Would you like to share pseudonymous usage data about this project with the Angular Team
at Google under Google's Privacy Policy at https://policies.google.com/privacy. For more
details and how to change this setting, see https://angular.dev/cli/analytics.

   No
Global setting: disabled
Local setting: No local workspace configuration file.
Effective status: disabled
✔ Which stylesheet system would you like to use? CSS             [ https://developer.mozilla.org/docs/Web/CSS                    ]
✔ Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? No
✔ Which AI tools do you want to configure with Angular best practices? https://angular.dev/ai/develop-with-ai None
CREATE student-information-dashboard/.prettierrc (161 bytes)
CREATE student-information-dashboard/README.md (1481 bytes)
CREATE student-information-dashboard/.editorconfig (314 bytes)
CREATE student-information-dashboard/.gitignore (622 bytes)
CREATE student-information-dashboard/angular.json (1972 bytes)
CREATE student-information-dashboard/package.json (806 bytes)
CREATE student-information-dashboard/tsconfig.json (957 bytes)
CREATE student-information-dashboard/tsconfig.app.json (429 bytes)
CREATE student-information-dashboard/tsconfig.spec.json (441 bytes)
CREATE student-information-dashboard/.vscode/extensions.json (130 bytes)
CREATE student-information-dashboard/.vscode/launch.json (470 bytes)
CREATE student-information-dashboard/.vscode/mcp.json (179 bytes)
CREATE student-information-dashboard/.vscode/tasks.json (978 bytes)
CREATE student-information-dashboard/src/main.ts (222 bytes)
CREATE student-information-dashboard/src/index.html (313 bytes)
CREATE student-information-dashboard/src/styles.css (80 bytes)
CREATE student-information-dashboard/src/app/app.css (0 bytes)
CREATE student-information-dashboard/src/app/app.spec.ts (696 bytes)
CREATE student-information-dashboard/src/app/app.ts (311 bytes)
CREATE student-information-dashboard/src/app/app.html (20144 bytes)
CREATE student-information-dashboard/src/app/app.config.ts (313 bytes)
CREATE student-information-dashboard/src/app/app.routes.ts (77 bytes)
CREATE student-information-dashboard/public/favicon.ico (15086 bytes)
✔ Packages installed successfully.
    Directory is already under version control. Skipping initialization of git.
krishnaik@Krishs-MacBook-Air Angular %
```

3. Folder Structure



• **angular.json**: it contains Angular workspace configuration, build settings, environments and assets.

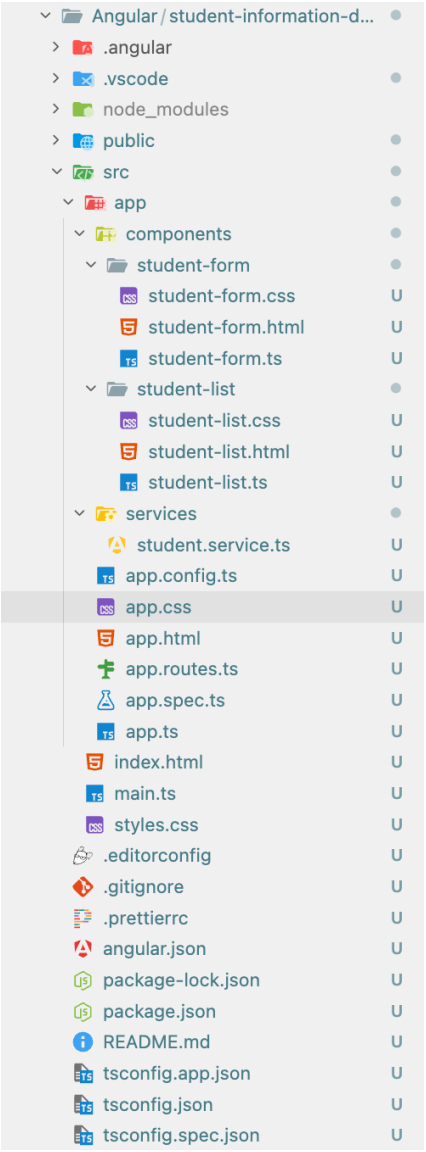
• **tsconfig.json**: TypeScript compiler configuration and transpilation rules.

• **src**: main application source which contains components, services, assets, routing.

• **main.ts**: Application bootstrapping starts here. Equivalent to main.jsx in react.

• **app**: equivalent to react's app and components.

4. Designing Our Project:

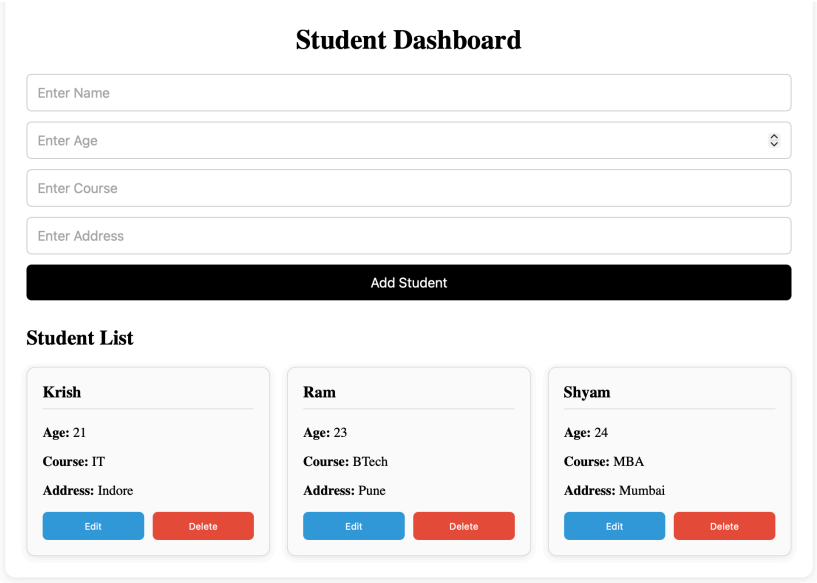


• **components:**
- **student-form:**
1. student-form.html
2. student-form.css
3. student-form.ts

- **student-list:**
1. student-list.html
2. student-list.css
3. student-list.ts

• **services**:
- student.service.ts

5. Project Screenshot:



6. Angular Folder Architecture in detail:

• **components**: It contains an individual component folder each with their own HTML, CSS, TypeScript and unit test cases.

• **services**: This Folder is responsible for handling all the services and API calls. Each service has a spec.ts file for testing purpose and a ts file in which our main functionality for service will be written.

• **directives**: This Folder is responsible for handling all the directives. Directives are simply and instruction to DOM. We can have custom as well as build-in Directives in Angular.

• **model**: This folder stores all the typescript model classes used to define data structure and business logic of an application.

• **core**: This folder serves as central storage hub of an angular application where you keep all important services, guards, interceptors for smooth functioning of the application.

• **shared**: This folder is necessary for organizing shared files and folders.

• **pages**: This folder serves as a directory to manage views and pages. It helps in organizing related components, template, styles ,etc. by keeping them under one folder.

Vue:

1. Need for Vue:

Before Vue:

- Applications were difficult to maintain and organize.
- State Synchronization issues.

Vue Introduced:

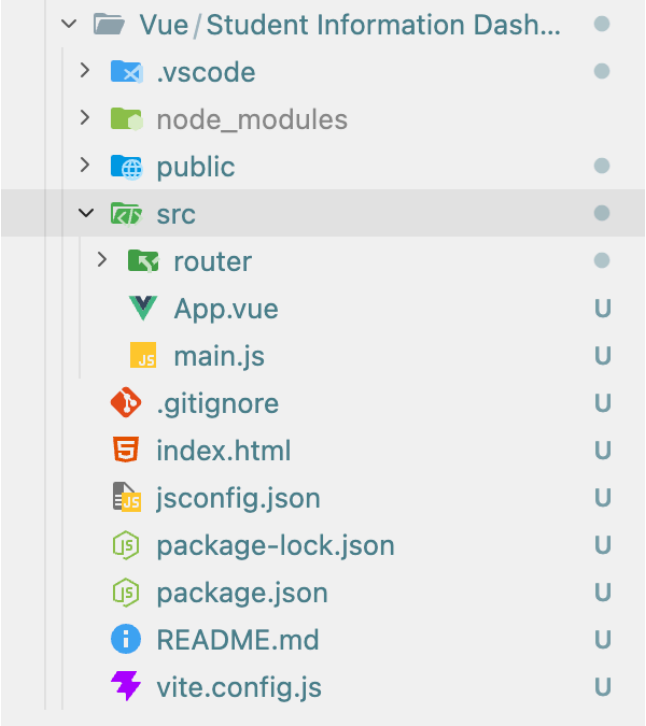
- Lightweight and simple syntax.
- progressive adoption and component system.

***Vue offers Angular like features and React like simplicity**

2. Creating Vue Project:

```
krishnaik@Krishs-MacBook-Air Vue % npm create vue@latest
Need to install the following packages:
create-vue@3.22.3
Ok to proceed? (y) y


> npx
> create-vue

┌  Vue.js - The Progressive JavaScript Framework
│
◇  Project name (target directory):
│  Student Information Dashboard
│
◇  Package name:
│  student-information-dashboard
│
◇  Use TypeScript?
│  No
│
◇  Select features to include in your project: (↑/↓ to navigate, space to select, a to toggle all, enter to confirm)
│  Router (SPA development)
│
◇  Select experimental features to include in your project: (↑/↓ to navigate, space to select, a to toggle all, enter to confirm)
│  none
│
◇  Skip all example code and start with a blank Vue project?
│  Yes

Scaffolding project in /Users/krishnaik/Downloads/IncubXperts Internship/Task 2/Vue/Student Information Dashboard...
│
└  Done. Now run:

   cd "Student Information Dashboard"
   npm install
   npm run dev

| Optional: Initialize Git in your project directory with:

   git init && git add -A && git commit -m "initial commit"
```

3. Folder Structure:

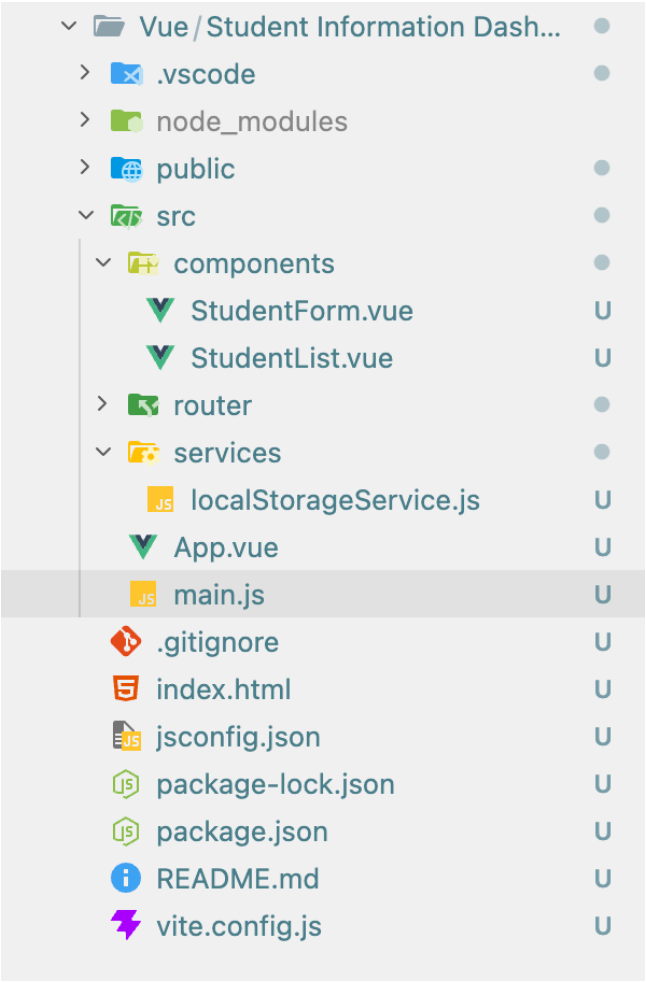


• **public:** static assets, same idea as react.

• **src**: contains the main application code. Includes components, App.vue, main.js.

• **App.vue**: this is the root component.

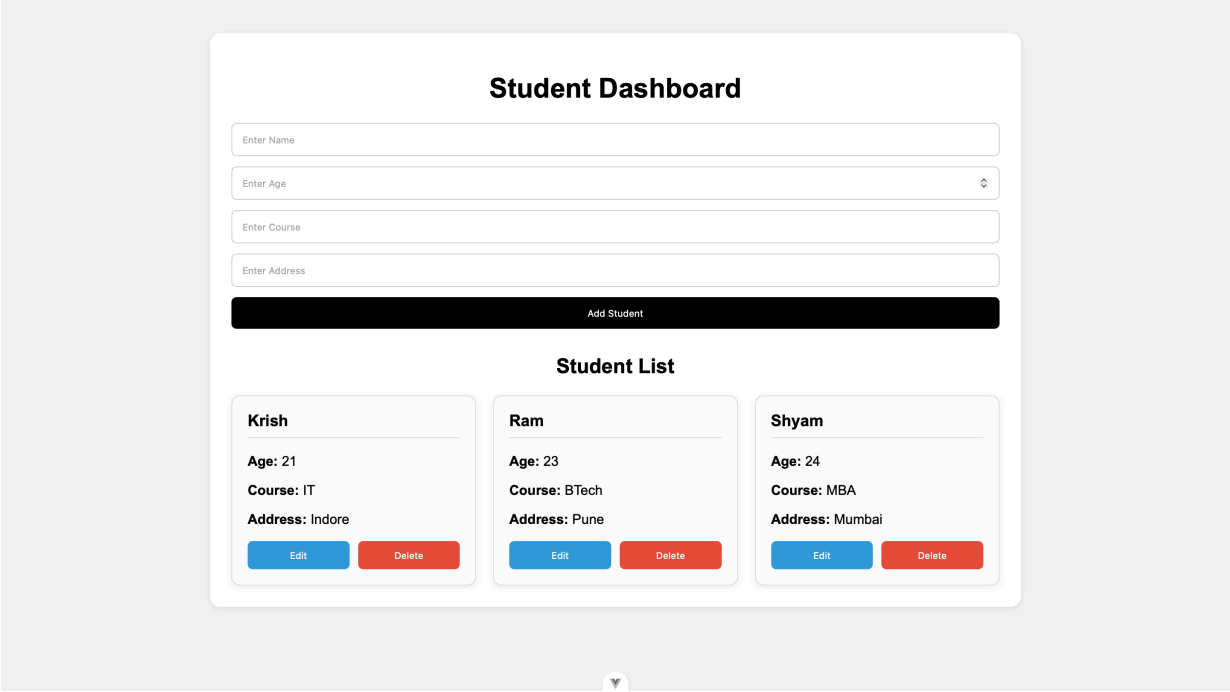
4. Designing Our Project:



• **components**:
-StudentForm.vue
-StudentList.vue

• **services**:
- localStorageService.js

5. Project Screenshot:



6. Vue Folder Architecture in detail:

```
/src
├── components
│   ├── ...
│
├── composables
│   ├── ...
│
├── services
│   ├── ...
│
├── utils
│   ├── ...
│
├── layouts
│   ├── ...
│
├── plugins
│   ├── ...
│
├── views
│   ├── ...
│
├── router
│   ├── ...
│
├── store
│   ├── ...
│
├── assets
│   ├── images
│   ├── styles
│
├── tests
│   ├── ...
│
├── App.vue
├── main.js
```

• components: Stores reusable UI components used throughout the application.

• composables: Contains reusable Composition API logic such as state handling or reactive functionality. Similar to React custom hooks

• services: Handles reusable business operations such as API calls, local storage handling, authentication, or external integrations.

• utils: Contains helper functions, validators, formatters, and utility methods used across multiple components.

• layouts: Defines reusable application layouts such as dashboard, admin, or authentication structures.

• plugins: Plugins can provide features such as notifications, themes, translations, or external library integration accessible throughout the application.

• views: Views typically combine multiple components and layouts to create complete screens displayed when navigating between application routes.

• router: Contains route configuration and navigation logic for the application.

• store: Manages centralized global state shared across multiple components.

• tests: Contains unit tests, integration tests, and component tests used to verify application functionality.

Comparative Study of React, Angular and Vue

1. Overview:

Feature	React	Angular	Vue
Type	UI Library	Full Framework	Progressive Framework
Developed By	Meta	Google	Evan You
Primary Language	JavaScript / JSX	TypeScript	JavaScript
Learning Curve	Medium	Difficult	Easy
Architecture Style	Flexible	Strict & Structured	Balanced
Routing	External Library	Built-in	Vue Router
State Handling	Hooks	Services	Reactive Data
Styling Method	Global / CSS Modules	Encapsulated Styles	Scoped Styles
Best Use Case	Dynamic UI Apps	Enterprise Applications	Lightweight Applications

2. Folder Structure Comparison:

Framework	Folder Structure Style	Observation
React	Flexible & Developer-Defined	Developers manually organize project folders according to application requirements.
Angular	Strict & Convention-Based	Angular follows predefined enterprise architecture and modular project organization.
Vue	Modular & Balanced	Vue provides scalable folder organization while maintaining development simplicity.

3. Component Architecture Comparison:

Framework	Component Structure	Explanation
React	Component.jsx	React combines UI and logic using JSX inside JavaScript files, giving developers maximum flexibility in component creation.
Angular	component.ts + component.html + component.css	Angular separates logic, template, and styling into different files for better maintainability and enterprise scalability.
Vue	Component.vue	Vue uses Single File Components where template, script, and style exist together inside one file for cleaner development.

4. Data Flow Comparison:

Framework	Data Flow	Important Detail
React	State → Virtual DOM → UI Update	React uses hooks and virtual DOM re-rendering to synchronize data changes with the interface.
Angular	Component → Change Detection → Template Update	Angular automatically tracks component changes using its built-in change detection mechanism.
Vue	Reactive Data → Dependency Tracking → DOM Update	Vue tracks reactive dependencies and updates only affected UI elements automatically.

5. Advantages and Limitations:

Framework	Advantages	Limitations
React	Flexible, reusable, large ecosystem	Requires additional libraries and manual architecture decisions
Angular	Enterprise-ready, strong architecture, complete framework	Steeper learning curve and larger boilerplate
Vue	Lightweight, beginner-friendly, simple syntax	Smaller ecosystem and lower enterprise adoption

6. Comparative Conclusion:

Framework	Final Observation
React	Best suited for developers preferring flexibility and custom frontend architecture decisions.
Angular	Most suitable for large-scale enterprise applications requiring strong structure and scalability.
Vue.js	Ideal for fast development, easier learning, and balanced frontend application architecture.